

NeuronSeqPerform Sonic Implementation Notes

Project Technical Documentation

August 8, 2026

1 Purpose

This document explains how audio is produced in NeuronSeqPerform from a sonic implementation perspective. It covers:

- timing and control-rate logic,
- LIF network activity generation and spike pooling,
- FM synthesis voice architecture and sample rendering,
- parameter modulation via MIDI (MPD218),
- practical behavior and constraints relevant to sound.

2 Signal-Flow Overview

Implementation files of interest:

- main loop: `main.py`
- LIF wrapper: `model/lif_network.py`
- synth wrapper: `model/fm_synth.py`
- native FM core: `synth_native/FMSynthMetal.mm`
- FM preset data: `model/presets.py`
- MIDI mapping: `control/midi_handler.py`

Per sequencer tick, the logic is:

1. Run N_{lif} micro-steps of LIF simulation.
2. Accumulate spike states over those micro-steps.
3. Downmix dynamic LIF grid to fixed synth grid 8×16 .
4. Trigger FM envelopes for current sequencer column.
5. Audio callback renders buffer using current FM synth state.

3 Clocking and Control Rate

3.1 Step Duration

The sequencer target step duration is a 16th-note at current tempo:

$$T_{\text{step}} = \frac{60}{\text{BPM}} \cdot \frac{1}{4} \quad (1)$$

Additional timing terms:

- swing offset for odd steps,
- random jitter scaled by quantization strength.

If quantization strength is $q \in [0, 1]$, jitter amplitude is proportional to $(1 - q)$.

3.2 Audio Buffering

Default audio settings from `config.py`:

- sample rate: 44100 Hz,
- block size: 1024 samples,
- stereo output by duplicating mono synth signal to both channels.

4 LIF Network as Sonic Driver

4.1 Purpose in this system

LIF activity is not directly audible. It controls FM operator envelopes via spike events.

4.2 Topologies and size

Supported topologies (native and fallback):

- Ring,
- FullyConnected,
- Feedforward,
- SparseRandom,
- SmallWorld.

Neuron count is runtime-modifiable (typically stepped among $\{128, 256, 512, 1024, 2048, 4096\}$).

4.3 Membrane update (fallback math)

The fallback LIF update follows a standard leaky form:

$$\Delta v = -(v - v_{\text{rest}}) + I \frac{\Delta t}{\tau} \quad (2)$$

with refractory masking and threshold/reset spike behavior.

Native LIF follows equivalent semantics with GPU implementation and explicit topology-driven connectivity.

4.4 Dynamic-grid to synth-grid mapping

Because synth rendering expects a fixed grid of operators (8×16), dynamic LIF activity is pooled:

- columns mapped by nearest-neighbor index selection,
- rows pooled by band-wise logical OR (max-pooling for spikes).

This keeps musical structure stable while allowing large or changing neuron fields.

5 FM Synth Architecture

5.1 Voice and operator structure

The synth uses 16 preset columns (voices) and 8 operators per voice. Operators are arranged in fixed carrier/modulator pairs:

$$(0 \leftarrow 1), (2 \leftarrow 3), (4 \leftarrow 5), (6 \leftarrow 7)$$

At runtime, only one sequencer column is the active target voice; transitions use gain ramps.

5.2 Per-operator frequency

For column c and operator o :

$$f_{c,o} = f_c^{\text{base}} \cdot r_{c,o} \cdot s_{\text{ratio}} \quad (3)$$

where:

- f_c^{base} is column base frequency (from root+scale mapping),
- $r_{c,o}$ is preset ratio,
- s_{ratio} is global ratio scale parameter.

5.3 Spike-gated envelopes

At tick transition, for current column c^* :

$$\text{env}_{o,c^*} \leftarrow \text{spike}_{o,c^*} \in \{0, 1\} \quad (4)$$

Then per audio buffer, carrier and modulator envelopes decay with different coefficients:

$$\text{env}_{\text{car}}^{\text{next}} = \alpha_{\text{car}} \text{env}_{\text{car}}, \quad \text{env}_{\text{mod}}^{\text{next}} = \alpha_{\text{mod}} \text{env}_{\text{mod}} \quad (5)$$

5.4 Per-pair FM sample equation

For pair p in a voice:

$$x_p[n] = A_p[n] \sin(\phi_c[n] + I_p[n] \sin(\phi_m[n])) \quad (6)$$

with:

$$A_p[n] \sim \text{level ramp from envelope} \quad (7)$$

$$I_p[n] \sim \text{mod index ramp} \times s_{\text{mod}} \quad (8)$$

Voice sample is average of active pairs, then weighted by voice gain ramp.

5.5 Voice crossfade and summation

Each of 16 voices has a per-buffer gain ramp from current to target state. Final pre-limiter sample:

$$y[n] = \sum_{v=1}^{16} g_v[n] \cdot \bar{x}_v[n] \quad (9)$$

5.6 Output soft limiting

Output is soft-clipped and scaled:

$$y_{\text{out}}[n] = \tanh(0.7 y[n]) \cdot V_{\text{master}} \quad (10)$$

6 Preset and Tonal Mapping

6.1 Presets

`model/presets.py` defines 16 presets. Each preset stores:

- ratios (8 values),
- levels (8 values),
- modulation indices (8 values).

6.2 Root and scale to base frequencies

Column base frequencies are generated from:

- root MIDI note,
- selected scale semitone offsets,
- degree and octave wrapping over 16 columns.

MIDI-to-frequency conversion:

$$f = 440 \cdot 2^{(m-69)/12} \quad (11)$$

7 MIDI Control Surface (MPD218)

7.1 Knobs (CC)

Bank	CC	Main sonic effect
A	3,9,12,13,14,15	10, master volume, active pairs, envelope decay speed, FM index scale, quantization strength
B	16,17,18,19,20,21	22, threshold, tau, weight scale, LIF drive, swing, FM ratio scale
C	22,23,24,25,26,27	28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 439, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 546, 547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689, 690, 691, 692, 693, 694, 695, 696, 697, 698, 699, 700, 701, 702, 703, 704, 705, 706, 707, 708, 709, 710, 711, 712, 713, 714, 715, 716, 717, 718, 719, 720, 721, 722, 723, 724, 725, 726, 727, 728, 729, 730, 731, 732, 733, 734, 735, 736, 737, 738, 739, 740, 741, 742, 743, 744, 745, 746, 747, 748, 749, 750, 751, 752, 753, 754, 755, 756, 757, 758, 759, 760, 761, 762, 763, 764, 765, 766, 767, 768, 769, 770, 771, 772, 773, 774, 775, 776, 777, 778, 779, 780, 781, 782, 783, 784, 785, 786, 787, 788, 789, 790, 791, 792, 793, 794, 795, 796, 797, 798, 799, 800, 801, 802, 803, 804, 805, 806, 807, 808, 809, 810, 811, 812, 813, 814, 815, 816, 817, 818, 819, 820, 821, 822, 823, 824, 825, 826, 827, 828, 829, 830, 831, 832, 833, 834, 835, 836, 837, 838, 839, 840, 841, 842, 843, 844, 845, 846, 847, 848, 849, 850, 851, 852, 853, 854, 855, 856, 857, 858, 859, 860, 861, 862, 863, 864, 865, 866, 867, 868, 869, 870, 871, 872, 873, 874, 875, 876, 877, 878, 879, 880, 881, 882, 883, 884, 885, 886, 887, 888, 889, 890, 891, 892, 893, 894, 895, 896, 897, 898, 899, 900, 901, 902, 903, 904, 905, 906, 907, 908, 909, 910, 911, 912, 913, 914, 915, 916, 917, 918, 919, 920, 921, 922, 923, 924, 925, 926, 927, 928, 929, 930, 931, 932, 933, 934, 935, 936, 937, 938, 939, 940, 941, 942, 943, 944, 945, 946, 947, 948, 949, 950, 951, 952, 953, 954, 955, 956, 957, 958, 959, 960, 961, 962, 963, 964, 965, 966, 967, 968, 969, 970, 971, 972, 973, 974, 975, 976, 977, 978, 979, 980, 981, 982, 983, 984, 985, 986, 987, 988, 989, 990, 991, 992, 993, 994, 995, 996, 997, 998, 999, 1000

7.2 Pads

- Bank A: root-note selection + velocity-dependent drive boost.
- Bank B: scale selection.
- Bank C: network functions (randomize/reset, topology cycling, neuron count stepping, etc.).

7.3 Aftertouch

Aftertouch is routed to one selected target (CC22 selects route), including threshold, tau, drive, tempo, quantization, swing, master volume, FM index scale, active pairs, or decay speed.

8 Native vs Fallback Rendering

8.1 Native (preferred)

Native FM synthesis uses Metal compute in `FMSynthMetal.mm`. The Python layer passes arrays and scalar parameters through pybind.

8.2 Fallback

A Python fallback path exists in `model/fm_synth.py` conceptually, but production behavior should be validated against your current branch state. In normal operation on macOS with compiled extension, native path is used.

9 Practical Sonic Notes

- LIF spikes do not generate audio directly; they gate FM envelopes.
- Increasing LIF steps/tick increases temporal density of envelope triggers.
- `active_pairs` controls spectral complexity by enabling 1..4 carrier-mod pairs.
- `mod_index_scale` and `ratio_scale` are primary timbre shapers.
- `decay_speed` changes per-buffer envelope decay and therefore articulation.
- Dynamic neuron-count changes can occur live; sequencer accumulation logic resets safely on shape changes.

10 Minimal Mental Model

For quick reasoning about the sound engine:

1. LIF generates a binary activity pattern each tick.
2. Pattern is pooled to 8×16 operator triggers.
3. Current sequencer column selects one FM voice.
4. Triggered operators start at `env=1` and decay.
5. FM pair sums are crossfaded, mixed, soft-limited, and output as mono-to-stereo.